SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
(Common to all Branches)
SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu

Mode of Exam
OFFLINE

Academic Year: 2023-24 (ODD)

RegNo

Test: CLAT-2 Date: 09-10-2023
Course Code & Title: 21CSS201T COA Duration: 100 minutes
Year & Sem: II & III SET D Max. Marks: 50

Course Articulation Matrix:

Course Learning Outcomes (CLO)	At the end of this course, learners will be able to:	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12
CO-2	Apply Boolean algebra as related to designing computer logic, through simple combinational and sequential logic circuits	H	M	-		-	-	-	-	-	-	-	-
CO-3	Examine the detailed operation of Basic Processing units and the performance of Pipelining	H	-	-		-	-	-	-	-	-	-	-

Part – A
(1 x 10 = 10 Marks)
Instructions: Answer all

Q. No	Questions	Marks	BL	CO	PO	PI Code
1	Which of the following is responsible for translating logical addresses to physical addresses? a) Memory Address Register (MAR) b) Memory Data Register (MDR) c) Arithmetic Logic Unit (ALU) d) Memory Management Unit (MMU)	1	1	2	1	2.1.2
2	Find the arithmetic left shift of 1 0 1 0 1 a) 0 1 0 1 0 b) 1 0 1 0 1 c) 0 1 0 1 1 d) 1 0 1 0 0	1	2	2	1	2.1.2
3	In 8086 microprocessor , the address bus is bit wide a) 12 Bit b) 10 Bit c) 16 Bit d) 20 Bit	1	1	2	1	2.1.2
4	Use DeMorgan's theorem to find the complement of A + BC a) $\overline{A}$ ($\overline{B}$ + $\overline{C}$) b) ABC c) A (B + C) d) A (BC)	1	1	2	1	2.1.2
5	Carry lookahead adder logic uses the concept of _____ a) Inverting the inputs b) Complementing the outputs c) Carry Propagate and Carry Generate d) Ripple factor	1	1	3	1	2.1.2

6	Which of the following multiplication algorithms is commonly used for signed multiplication? a) Booth's algorithm b) Wallace tree algorithm c) Folding algorithm d) Radix-4 algorithm	1	1	3	1	2.1.2
7	Which of the following statements is true for carry-save addition of summands? a) It can be performed in a single cycle b) It requires fewer logic gates than carry-propagate addition c) It always produces the final sum result directly d) It is only used for fixed-point arithmetic	1	1	3	1	2.1.2
8	Which division algorithm is most commonly used for binary division operations? a) Restoring division b) Non-restoring division c) Radix-4 division d) Booth's division	1	1	3	1	2.1.2
9	What are the three components of an IEEE 754 floating-point number representation? a) Sign exponent and mantissa b) Sign, integer part and fractional part c) Mantissa, exponent and bias d) Sign, exponent and base	1	2	3	1	2.1.2
10	Which format is used for double precision floating-point numbers in IEEE 754 a) 64 Bits b) 32 Bits c) 16 Bits d) 128 Bits	1	1	3	1	2.1.2

RegNo														

Test: CLAT-2 Date: 09-10-2023

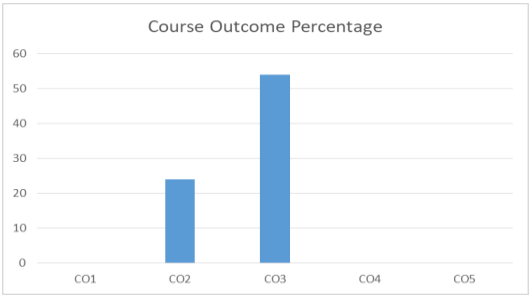
Course Code & Title: 21CSS201T COA Duration: 100 minutes

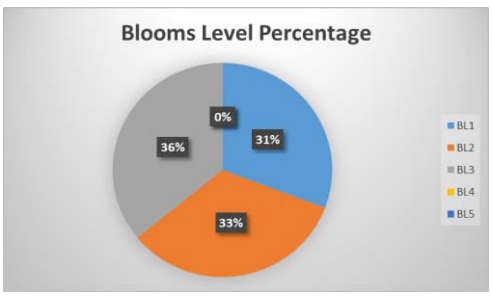
Year & Sem: II & III SET D Max. Marks: 50

	Part – B (4*4 = 16 Marks) Answer any four questions					
11	Explain the steps for multiplying 9 and 12 using a shift-and-add multiplier method?	4	2	2	2	2.1.2
12	What are the key distinctions between microprocessors and microcontrollers in terms of their functions and applications	4	2	2	1	2.1.2
13	What are the components and connections needed to construct a full adder using two half adders?	4	2	3	1	2.1.2
14	Describes how to negate the union and intersection of sets or logical conditions	4	1	3	1	2.1.2
15	Evaluate X = (A + B) * (C + D) using One Address Instruction	4	3	3	1	2.1.2
	Part – C (2*12 = 24 Marks)					
16	a) Demonstrate the steps involved during Instruction execution and straight line sequencing with examples [OR] b) Raghav is designing a 4-bit circuit for his digital electronics project. He has already connected four full-adder units in series, and now he needs to implement the mode control functionality. The mode control input, labeled "M" will be controlled by a switch. Raghav wants to ensure that his circuit can handle both addition and subtraction operations. He sets up a scenario to test the functionality.	12 12	1 2	2 3	2 1	2.1.2 2.1.2
17	a) John is studying computer architecture and learning about the Booth Multiplication algorithm. He aims to use this algorithm to multiply -15 and -13. He follows the steps meticulously to compute the result and also creates a neat flowchart to facilitate the multiplication process. [OR] b) i) Find 8 ÷ 3 using Restoring Division (6) ii) Find +13 and -6 using Bit Pair Recoding of Multipliers (6)	12 12	3 3	3 3	1 1	2.1.2 2.1.2

*Performance Indicators are available separately for Computer Science and Engineering in AICTE examination reforms policy.

Course Outcome (CO) and Bloom's level (BL) Coverage in Questions





Approved by the Audit Professor/Course Coordinator

SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
(Common to all Branches)

Mode of Exam
OFFLINE

DEPARTMENT OF COMPUTING TECHNOLOGIES

SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu

---

ANSWER KEY

Part – A

1. D)Memory Management Unit (MMU)
2. A) 0 1 0 1 0
3. D) 20 Bit
4. A) **A (B + C)**
5. C) Carry Propagate and Carry Generate
6. A) Booth's algorithm
7. B) It requires fewer logic gates than carry-propagate addition
8. A) Restoring division
9. A) Sign exponent and mantissa
10. A) 64 Bits

PART B

11. **Explain the steps for multiplying 9 and 12 using a shift-and-add multiplier method?**

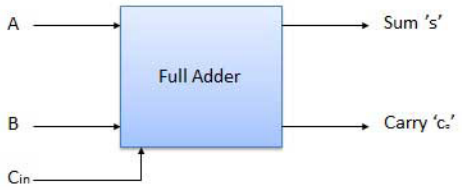
Step	A	Q	B	Operation
0	0000	1100	1001	Initialization
1	0000	0110	1001	Shift right A_Q
2	0000	0011	1001	Shift right A_Q
3	1001	0011	1001	Add B to A
	0100	1001	1001	Shift right A_Q
4	1101	1001	1001	Add B to A
	0110	1100	1001	Shift right A_Q

12. **What are the key distinctions between microprocessors and microcontrollers in terms of their functions and applications**

Sl No	Microprocessor	Microcontroller
1	CPU is stand-alone, RAM, ROM, I/O, timer are separate	CP,RAM,ROM,I/O and timer are all on single chip
2	Designer can decide on the amount of ROM,RAM and I/O ports	Fix amount of on-chip ROM, RAM, I/O ports
3	Expansive	Not Expansive
4	General purpose	Single purpose
5	Microprocessor based system design is complex and expensive	Microcontroller based system design is rather simple and cost effective
6	The instruction set of microprocessor is complex with large number of instructions.	The instruction set of a Microcontroller is very simple with the less number of instructions.

13. **What are the components and connections needed to construct a full adder using two half adders?**

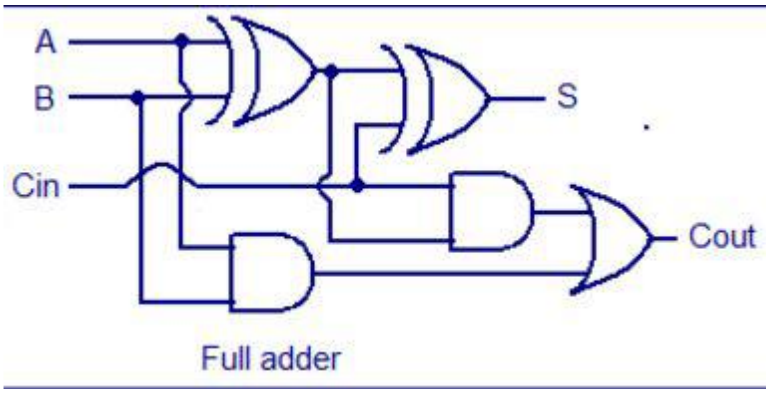
Full adder is developed to overcome the drawback of Half Adder circuit. It can add two one-bit numbers A and B, and carry c. The full adder is a three input and two output combinational circuit.



$$S = A\overline{B}\overline{C} + \overline{A}\,\overline{B}C + ABC + \overline{A}B\overline{C}$$
$$= C\,(AB + \overline{A}\,\overline{B}) + \overline{C}\,(\overline{A}B + A\,\overline{B})$$
$$= C\,(\overline{\overline{A}B + A\,\overline{B}}) + \overline{C}\,(\overline{A}B + A\,\overline{B})$$
$$= C\,(\overline{A \oplus B}) + \overline{C}\,(A \oplus B) \;= A \oplus B \oplus C$$

$$C_{out} = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$
$$= (\overline{A}B + A\overline{B})\,C + AB\,(\overline{C} + C)$$
$$= (A \oplus B).C + AB.$$

Inputs			Output	
A	B	Cin	S	Co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



14. De Morgan's theorem are used to solve the expressions of Boolean Algebra.

• The theorem explains that the complement of the product of all the terms is equal to the sum of the complement of each term. Likewise, the complement of the sum of all the terms is equal to the product of the complement of each term.

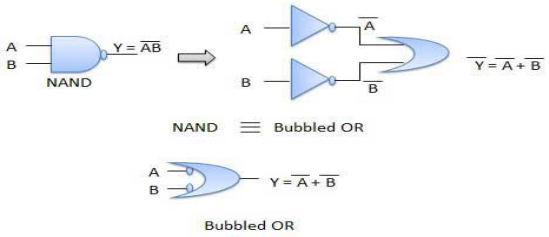
Theorem1

$$\overline{A.B} = \overline{A} + \overline{B}$$

NAND = Bubbled OR

• The left hand side (LHS) of this theorem represents a NAND gate with inputs A and B, whereas the right hand side (RHS) of the theorem represents an OR gate with inverted inputs.

• This OR gate is called as **Bubbled OR**.



A	B	$\overline{AB}$	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

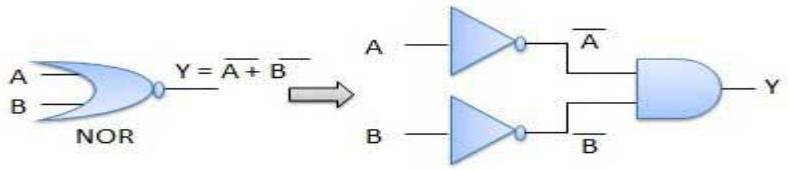
Theorem 2:

$$\overline{A + B} = \overline{A}.\overline{B}$$

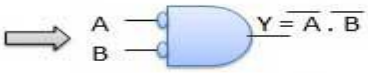
NOR = Bubbled AND

The LHS of this theorem represents a NOR gate with inputs A and B, whereas the RHS represents an AND gate with inverted inputs.

• This AND gate is called as **Bubbled AND**



NOR ≡ Bubbled AND



Bubbled AND

A	B	$\overline{A+B}$	$\overline{A}$	$\overline{B}$	$\overline{A}$. $\overline{B}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

15. **Evaluate X = (A + B) * (C + D) using One Address Instruction**

Using One-Address instruction

1. LOAD A ; AC ← M[A]
2. ADD B ; AC ← AC + M[B]
3. STORE T ; M[T] ← AC
4. LOAD C ; AC ← M[C]
5. ADD D ; AC ← AC + M[D]
6. MUL T ; AC ← AC * M[T]
7. STORE X ; M[X] ← AC

16. **A) Instruction execution** needs the following steps, which are

• PC (program counter) register of the processor gives the address of the instruction which needs to be fetched from the memory.

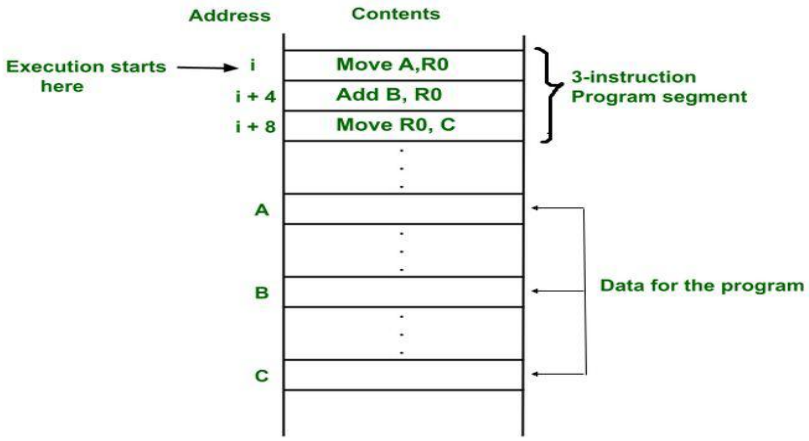
• If the instruction is fetched then, the instruction opcode is decoded. On decoding, the processor identifies the number of operands. If there is any operand to be fetched from the memory, then that operand address is calculated.

• Operands are fetched from the memory. If there is more than one operand, then the operand fetching process may be repeated (i.e. address calculation and fetching operands).

• After this, the data operation is performed on the operands, and a result is generated.
• If the result has to be stored in a register, the instructions end here.
• If the destination is memory, then first the destination address has to be calculated. Then the result is then stored in the memory. If there are multiple results which need to be stored inside the memory, then this process may repeat (i.e. destination address calculation and store result).
• Now the current instructions have been executed. Side by side, the PC is incremented tocalculate the address of the next instruction.
• The above instruction cycle then repeats for further instructions.

Straight line sequencing

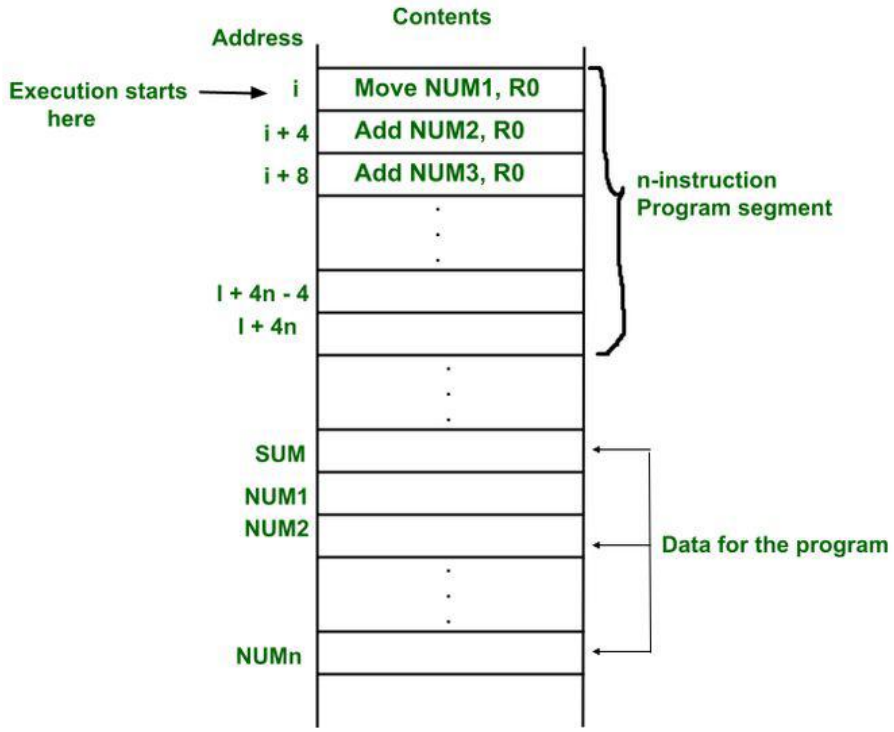
• Straight line sequencing means the instruction of a program is executed in a sequential manner(i.e. every time PC is incremented by a fixed offset).
• And no branch address is loaded on the PC.



Example 1:

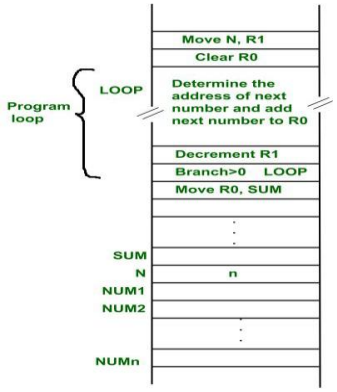
•programs and data are stored in the same memory, i.e. von Neumann architecture.
•First instruction of a program is stored at address i.
•PC gives address i and instruction stored at that address i is fetched from the memory and then decoded and then operand
•A is fetched from the memory and stored in a temporary register and then the instruction is executed(i.e. content of address A is copied into processor register R0).
•the PC gets incremented by 4(i.e. it contains the address of the next instruction) because the instruction and memory segment is of 4 bytes.
•So the instruction at address i is executed.
•So every time, the PC is incremented by 4.

Therefore, the program is executing in a sequential manner. And this process is called straight line sequencing.



The addresses of the memory locations containing the n numbers are represented as NUM1,NUM2…..NUMn(i.e. NUM1 address includes first number).

•The first number is stored into processor register R0. And every other number is added to register R0. Finally, when the program ends(i.e. n numbers are added, the result is placed in memory location SUM



The second way is to use a loop to add n number. But here straight line sequencing is not used because every time loop iteration ends, PC has to load the branch address and program execution starts from that address.

•Here the location N stores the value of n.

•Processor register R1 is used as a counter to determine the number of times the loop gets executed.

•The contents of the location N are moved into R1 at the start of program execution.

•After that, register R0 is cleared.

•The address LOOP is reloaded again and again until R1 becomes 0.

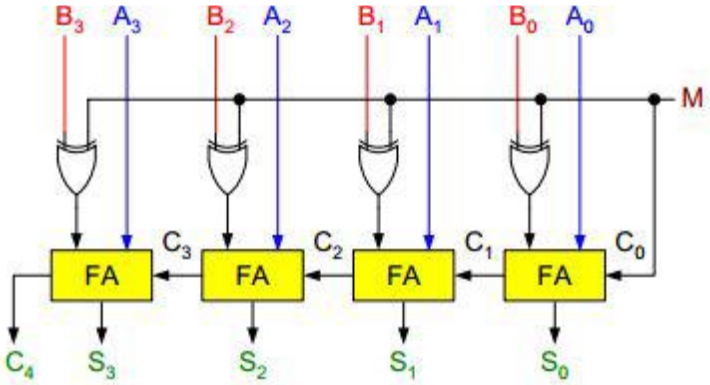
•Every time a number is added, then the R1 value is decremented.

•When R1 becomes 0, we come out of the loop and the result which is stored at R1 is copied into memory location SUM.

B) **Binary Parallel Adder/Subtractor:**

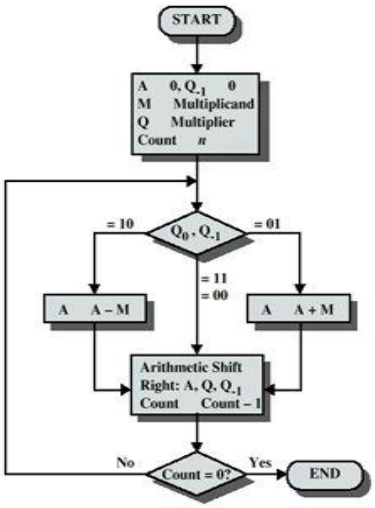
•The addition and subtraction operations can be done using an Adder-Subtractor circuit.

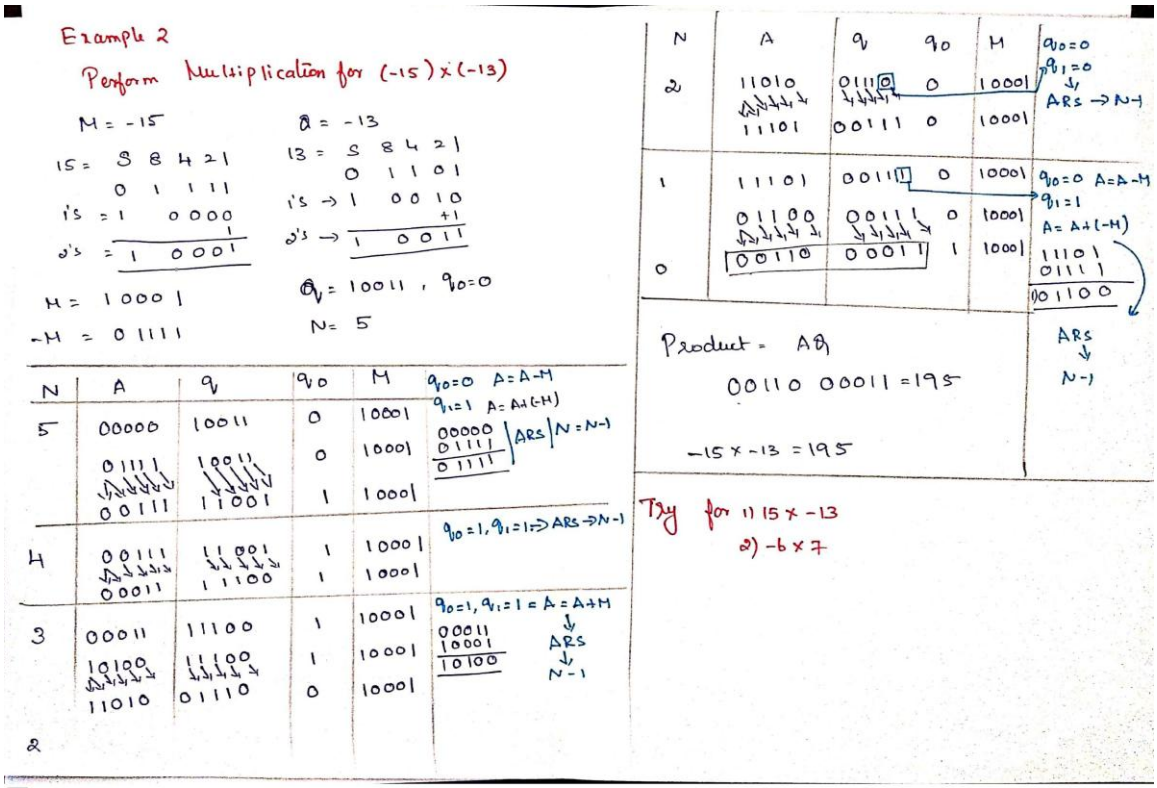
•The figure shows the logic diagram of a 4-bit Adder-Subtractor circuit.



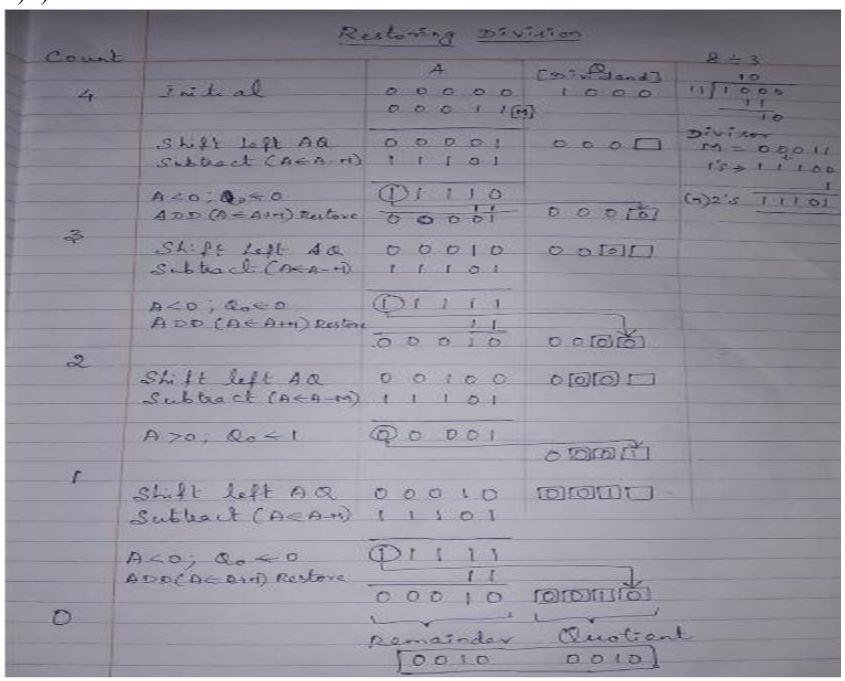
The circuit has a mode control signal M which determines if the circuit is to operate as an adder or a subtractor.
•Each XOR gate receives input M and one of the inputs of B, i.e., Bi. To understand the behavior of XOR gate consider its truth table given below.
If one input of XOR gate is zero then the output of XOR will be same as the second input. While if one input of XOR gate is one then the output of XOR will be complement of the second input.
So when M = 0, the output of XOR gate will be Bi + 0 = Bi. If the full adders receive the value of B, and the input carry C0 is 0, the circuit performs A plus B.
• When M = 1, the output of XOR gate will be Bi + 1 = Bi. If the full adders receive the value of B', and the input carry C0 is 1, the circuit performs A plus 1's complement of B plus 1, which is equal to A minus B.

17.A)




Example 2
Perform Multiplication for (-15) x (-13)
M = -15
15 = 8 4 2 1
0 1 1 1 1
1's = 1 0 0 0 0
+1
2's = 1 0 0 0 1
M = 1 0 0 0 1
-M = 0 1 1 1 1
Q = -13
13 = 8 4 2 1
0 1 1 0 1
1's → 1 0 0 1 0
+1
2's → 1 0 0 1 1
Q = 10011 , q0 = 0
N = 5
N A q q0 M
2 11010 01110 0 10001
11101 00111 0 10001
1 11101 00111 0 10001
01100 00111 0 10001
0 00110 00011 1 10001
Product = AQ
00110 00011 = 195
-15 x -13 = 195
Try for 1) 15 x -13
2) -6 x 7
N A q q0 M
5 00000 10011 0 10001
01111 10011 0 10001
00111 11001 1 10001
4 00111 11001 1 10001
00011 11100 1 10001
3 00011 11100 1 10001
10100 11100 1 10001
11010 01110 0 10001
2

B) i)


Restoring Division
Count
A
[Dividend]
8 ÷ 3
4 Initial 0 0 0 0 0 1 0 0 0
0 0 0 1 1 [M]
Shift left AQ 0 0 0 0 1 0 0 0 □
Subtract (A<A-M) 1 1 1 0 1
A<0; Q0<0 1 1 1 1 0
ADD (A<A+M) Restore 0 0 0 1 1 0 0 0 0
3
Shift left AQ 0 0 0 1 0 0 0 □
Subtract (A<A-M) 1 1 1 0 1
A<0; Q0<0 1 1 1 1
ADD (A<A+M) Restore 0 0 0 1 0 0 0 0
2
Shift left AQ 0 0 1 0 0 0 □
Subtract (A<A-M) 1 1 1 0 1
A>0; Q0<1 0 0 0 0 1
1
Shift left AQ 0 0 0 1 0
Subtract (A<A-M) 1 1 1 0 1
A<0; Q0<0 1 1 1 1
ADD (A<A+M) Restore 0 0 0 1 0
0
Remainder
0 0 1 0
Quotient
0 0 1 0
Division
M = 0 0 0 1 1
1's → 1 1 1 0 0
+1
(-)2's 1 1 1 0 1

ii)

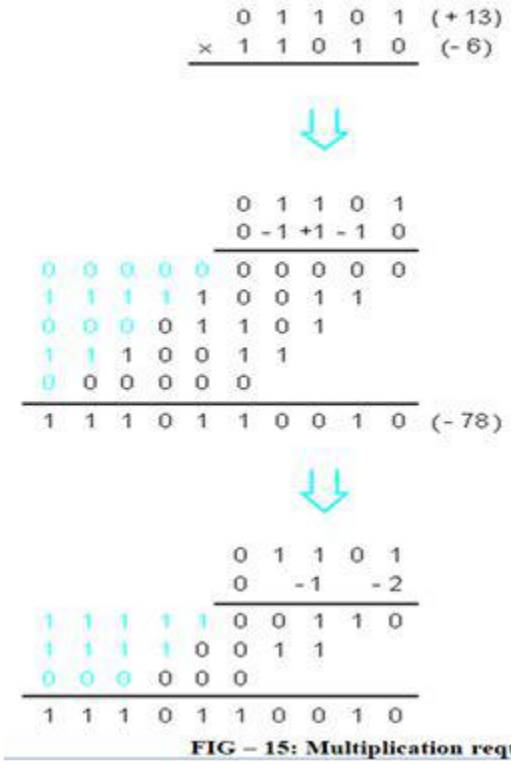


FIG – 15: Multiplication requiring n/2 summands.